Module 03 — Asking for what you want

Claude Code 101 · Beginner Workshop · Module 3 of 8

The shortest path from "I have a question" to "I have a useful answer" is a clear first prompt.

What you'll learn

By the end of this 30-minute lesson you will be able to:

1. Write a prompt that names a **role**, a **goal**, and at least one **constraint**.
2. Tell when a vague prompt needs to be rewritten instead of followed up.
3. Pick the right output format up front (bullets, table, code, one sentence).

Why this matters

- A 20-second investment in the first prompt usually saves three follow-ups.
- "Make my code better" is the prompt people most often regret. There is no single answer to it. A specific prompt gets a specific answer you can actually act on.
- Beginners over-rely on follow-ups. Experienced users front-load the prompt. That habit shift is the entire content of this lesson.

The one concept

> **Role + Goal + Constraint + Format = a prompt you won't have to rewrite.**

- **Role**: who Claude should pretend to be — "act as a careful tester", "act as a Python beginner's tutor".
- **Goal**: the single thing you want — "find a bug", "explain this function".
- **Constraint**: a hard limit — "≤ 50 words", "no jargon", "do not change file names".
- **Format**: how the answer should look — bullets, a table, a code block, one sentence.

Miss any of the four and Claude has to guess. Guessing is where bad answers come from.

Show me

Compare two prompts about the same code review request.

Vague (typical first attempt):

```
> Can you review my code?
```

Claude has no file, no language, no goal, no audience. The reply will be generic.

Clear (role + goal + constraint + format):

```
> Act as a careful Python reviewer. I will paste a 30-line function below. Your goal: find at most 3 real bugs, in order of severity. Constraint: no style nits and no rewrites. Format: a numbered list, each entry < 15 words.
```

Same model, same code — the second prompt gets a focused, actionable list every time.

Try it yourself

Pick a real ~30-line function from any project you have (or use the sample in `exercises/beginner/part-03/starter/`). Then write **one** prompt using the role + goal + constraint + format pattern.

Save the prompt and Claude's reply into `~/sharp-prompt.txt`. The exercise README has the full step-by-step.

Time budget: 12 minutes (most of it is reading the function and writing the prompt; the model reply is fast).

Common mistakes

- **Skipping the role.** "Find bugs" is fine, but "act as a careful reviewer who only flags real bugs" is much more focused.
- **Setting goal but no format.** You get a wall of prose when you wanted a list.
- **Stacking too many constraints.** Three is usually enough. Ten contradict each other.
- **Asking for everything at once.** "Find bugs AND rewrite AND explain AND write tests" is four prompts in a trench coat. Pick one.
- **Saying "be concise"** without a number. "≤ 50 words" works; "concise" does not.

Lesson reflection

Take 90 seconds:

1. Which of the four pieces (role, goal, constraint, format) did you most often skip?
2. Did your "clear" prompt get a usable answer on the first try? If not, which piece was still missing?
3. Pick one prompt you've used in real life recently. Could you have saved a follow-up by adding a constraint to it?

What's next

Module 04 — **Reading code together** — uses the prompt shape you just learned to ask Claude to explain code you didn't write. Same skill, different goal.

Budget for Module 04: 25 minutes.

Glossary card

- **Constraint**: A limit you put on Claude's reply, such as a word count or a list of words it must avoid.
- **Prompt**: The text you send to Claude. One message in the conversation.
- **Role prompt**: A prompt that tells Claude what role to play, for example "Act as a careful junior tester.".